



SHRI VILEPARLE KELAVANI MANDAL'S



SHRI BHAGUBHAI MAFATLAL POLYTECHNIC

‘Lan Based File Sharing App’

Diploma

In

Computer Engineering

Submitted by:

1. Divesh Bangera (A005)
2. Dhruven Gosia (A017)
3. Dev Mandora (A025)

Under the guidance of:

Guide name:
Mrs. Pradnya Natekar

Computer Engineering Department

(2025-2026)

This is to certify that

Mr. Divesh Bangera, Mr. Dhruven Gosia, Mr. Dev Mandora

Roll no. A005, A017, A025

has satisfactorily presented and submitted the Synopsis of

‘Lan Based File Transfer System’

Guide

H.O.D



SHRI VILEPARLE KELAVANI MANDAL'S

SHRI BHAGUBHAI MAFATLAL POLYTECHNIC



VISION

Create a sustainable academic environment to produce highly competent Computer Professionals.

MISSION

1. To expose students to latest tools and technologies in computing.
2. To keep faculties in tune with current professionals demands by adequate training and higher studies.
3. To adapt to rapid advancements in the computing by engaging students in the lifelong learning.
4. To incorporate some ethical moral and social values amongst students for benefit of the Society.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The Program Education Objectives (PEOs) are to produce Computer Professionals

1. Able to identify, frame and solve computing problems by applying their knowledge in Computer Engineering.
2. Equipped with knowledge and skill sets to get employed in field of Computer Engineering.
3. Capable of pursuing Higher Studies.

PROGRAM SPECIFIC OUTCOME (PSO)

PSO1. Ability to Demonstrate basic knowledge of a Computer Networking, Microprocessor based systems, Database Management System and Operating System for Computer based Application.

PSO2. Ability to demonstrate an application-oriented approach to enhance student's career by acquiring professional and problem-solving skill.



SHRI VILEPARLE KELAVANI MANDAL'S

SHRI BHAGUBHAI MAFATLAL POLYTECHNIC



Course Outcomes

Program – CSE

Course – Project

Sem – V

Course Code – PRO230812

CO No.	COURSE OUTCOMES
CO1	Identify problem definitions that can be addressed by applying the acquired knowledge & skill
CO2	Plan the activities with timeline chart, work in groups/team and co-ordinate the work
CO3	Select design methodologies & its implementation.
CO4	Write the Project Report

TABLE OF CONTENTS:

Sr.no	Contents	Page Number
1.	Abstract	5
2.	Introduction	5
3.	Problem Statement	5
4.	Proposed Solution	6
5.	System Flowchart	7
6.	Project Planning	8
7.	System Modules	9
8.	System Requirements	10
9.	Limitations	10
10.	Future Scope	10
11.	Conclusion	11
12.	References	11

ABSTRACT:

In situations where internet access is unavailable or where sensitive data must remain within a closed network, there is a clear demand for a secure and lightweight communication system that works seamlessly across different types of devices. This project proposes the development of **Lanvan**, a browser-based LAN application designed to run on any system capable of hosting a local Python server. The application will support encrypted file transfers, real-time chat, clipboard sharing, and optional quick device pairing using QR codes. All of these features function entirely offline over a local Wi-Fi or hotspot connection. By removing the need for internet access or platform-specific apps, Lanvan aims to provide a flexible and dependable communication platform suited for secure environments, on-site collaboration, and temporary offline operations.

INTRODUCTION:

There are many situations where internet access is either unavailable, restricted, or intentionally avoided. Whether due to strict security policies, isolated work environments, or temporary setups in the field, reliable and private local communication becomes a clear necessity.

Yet, most modern tools rely heavily on cloud services and external servers, making them unsuitable for these scenarios. Many existing solutions are not designed for offline use. They often require constant connectivity, lack cross-device flexibility, or fail to provide secure peer-to-peer communication within a local area network.

To solve this, the project proposes Lanvan, a lightweight, browser-based communication system built for offline environments. It uses Python's Flask framework for its simplicity, speed, and ability to run efficiently on minimal hardware. Real-time messaging is handled through **WebSocket technology**, allowing secure file sharing, chat, and clipboard transfer — all without the need for internet or third-party services. The system is accessible through any modern web browser over a local Wi-Fi or hotspot connection.

PROBLEM STATEMENT:

Whether it's securely sharing files in an office, sending documents between laptops at home, or coordinating tasks in a temporary field setup, local communication is often necessary. These needs arise not only in secure or professional environments but also in casual everyday situations where fast and private sharing matters.

However, current methods fall short. Physical tools like USB drives and data cables can be slow, get lost, or even cause damage if corrupted. They also suffer from port compatibility issues—like trying to connect a Type-A device to a Type-C port. To avoid these issues, users often turn to cloud platforms or email, even in places where internet access is limited or discouraged. This creates privacy risks, especially if users forget to log out of shared devices.

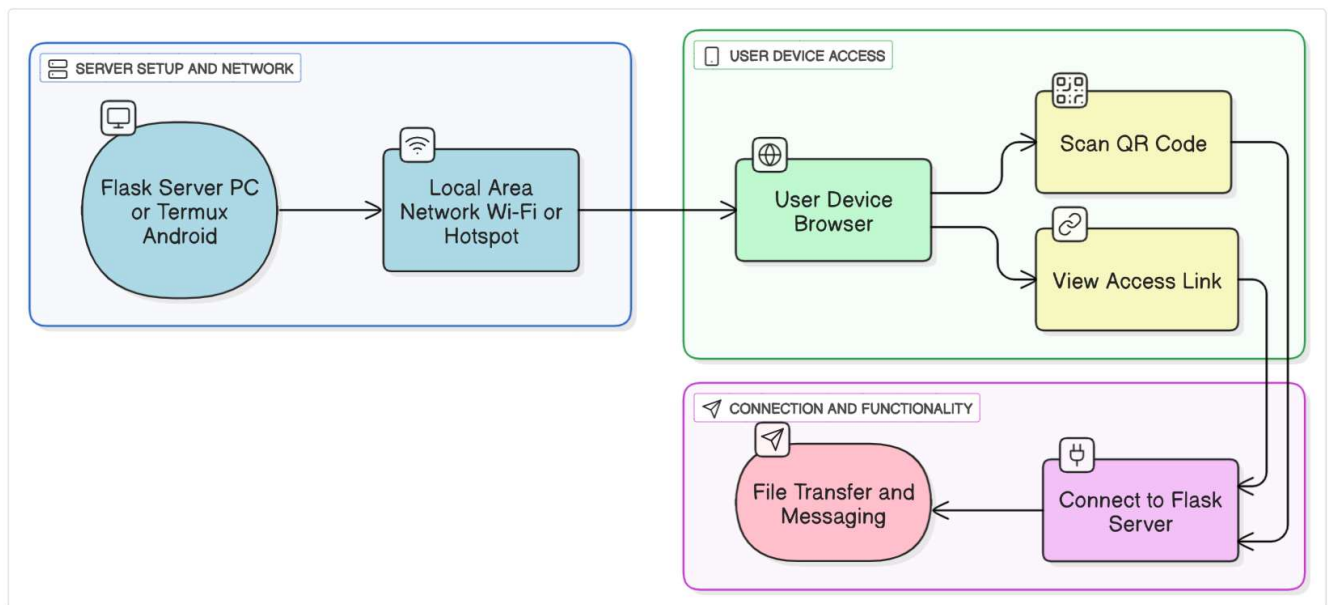
Relying on either physical storage or online services makes local file sharing inefficient and sometimes unsafe. There's a clear need for a simple, secure, and device-friendly system that works entirely within a local network — no internet, no cables — just fast, direct communication that works across devices and use cases

PROPOSED SOLUTION:

To overcome the drawbacks of physical file sharing and internet-dependent platforms, Lanvan enables real-time peer-to-peer encrypted file transfer, secure messaging, clipboard sharing, and fast device pairing via QR codes. Designed with an offline-first approach, all data stays within the local network, eliminating risks associated with cloud platforms.

The system consists of a Flask-based backend serving a responsive web UI accessible from any device browser. On Android, the server runs in Termux and can be launched via shortcuts for seamless access—no dedicated mobile app is required.

Users connect to the Flask server by either manually entering the displayed URL or scanning a QR code. Once connected, they can exchange files, messages, and clipboard data securely and efficiently.

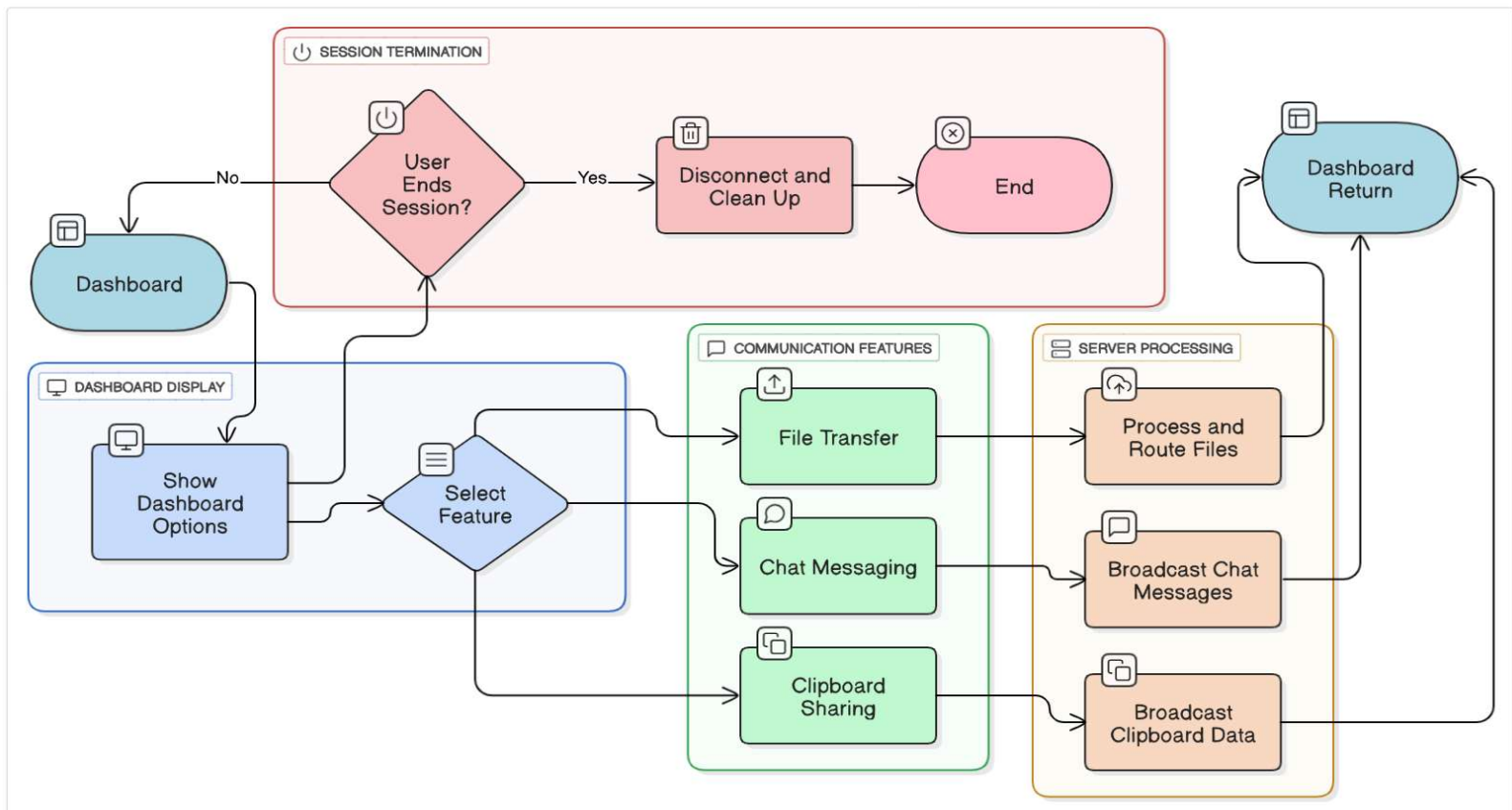
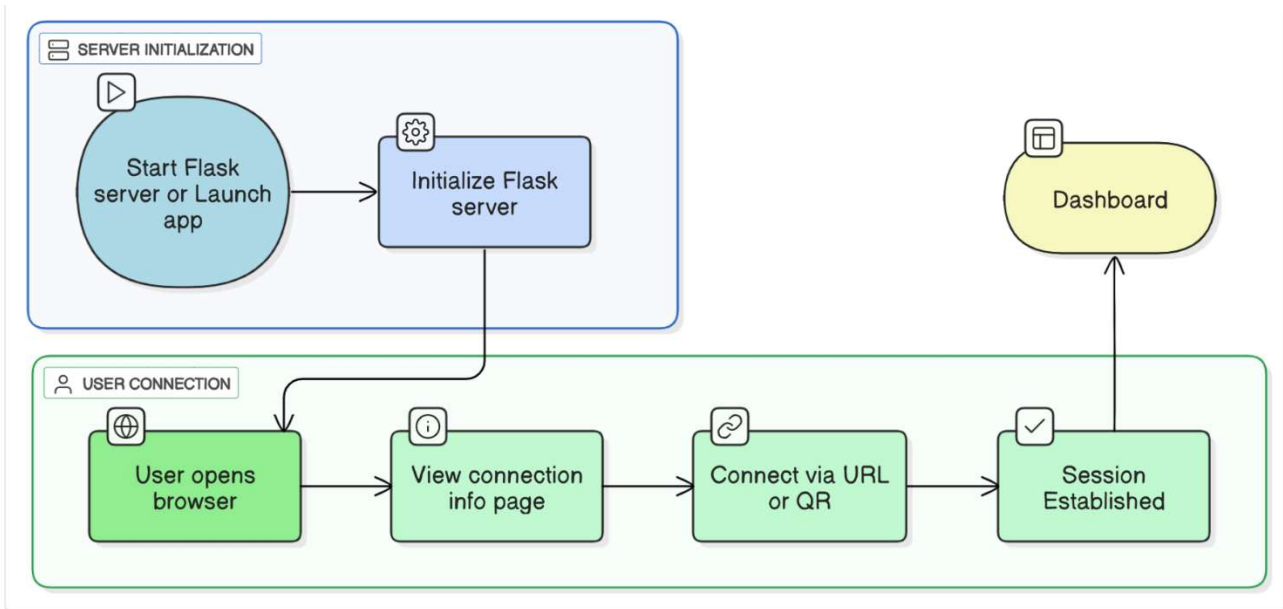


Typical USB 3.0 transfers reach speeds around 200–400 MB/s, and cloud transfers over average internet connections range from 2–10 MB/s. Lanvan aims to deliver transfer speeds between **10–20 MB/s** over standard Wi-Fi LAN, providing a fast and private alternative for local communication.

Lanvan will support both HTTP and HTTPS connections, with HTTPS providing acceptable encryption for secure communication. Future development will focus on implementing stronger end-to-end encryption methods, aiming to enhance data privacy while maintaining optimal transfer speeds.

This project follows the **Iterative Model** of software development. Features are implemented in progressive cycles, with each iteration focusing on a specific set of functionalities such as file transfer, QR code connectivity, real-time chat and encryption. This approach allows for continuous refinement, user testing, and easier integration of optional features. A fully usable version of the core application will be achieved by **Phase 2** (refer Project Planning Pg. 8), after which further enhancements and optional features will be integrated in later phases.

SYSTEM FLOWCHART:



PROJECT PLANNING:

The project is planned over 16 weeks and divided into four progressive phases. Each phase focuses on a core set of tasks, with continuous testing and gradual GUI development. Basic functionality is completed early, and advanced features and polish are added in later weeks.

Phase 1: Core Flask App (Weeks 1–5)

Initial setup of the Flask server, backend routes, and basic features like file upload/download, clipboard sharing, and QR code connection. A functional GUI is started alongside backend development. Features are tested individually, with a mid-project cross-device test in week 5.5.

Phase 2: Termux Automation & Android Integration (Weeks 6–10)

Focus shifts to running the server on Android using Termux. Integration with automation tools like MacroDroid or Termux Widget enables one-tap startup and browser launch. UI optimizations and reliability testing ensure smooth operation across mobile devices. By the end of this phase, a fully usable version of the core application will be ready

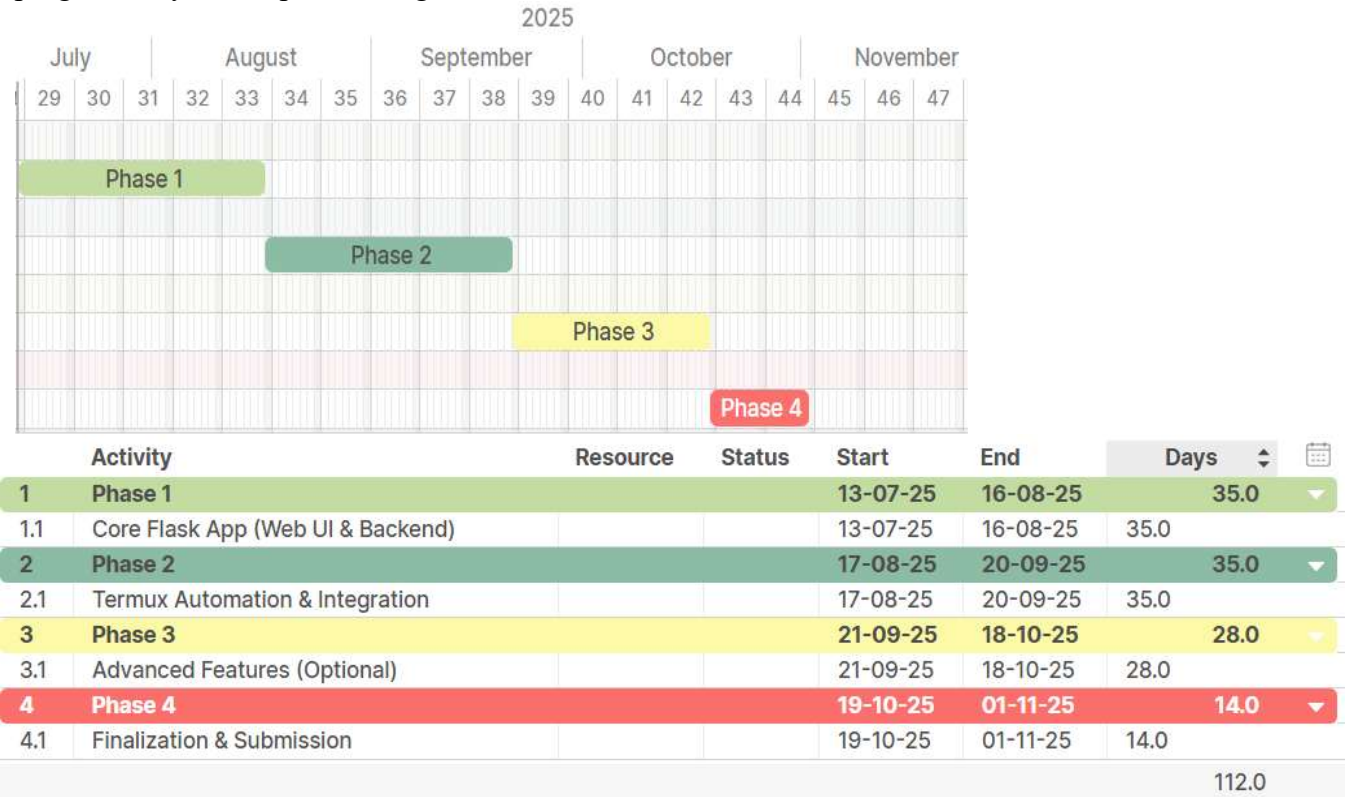
Phase 3: Advanced Features (Optional) (Weeks 11–14)

Optional add-ons such as real-time LAN chat, bulk file upload, smart QR codes with metadata, and AES-256 encryption are implemented. These features are modular and independently tested. The GUI continues to improve during this phase.

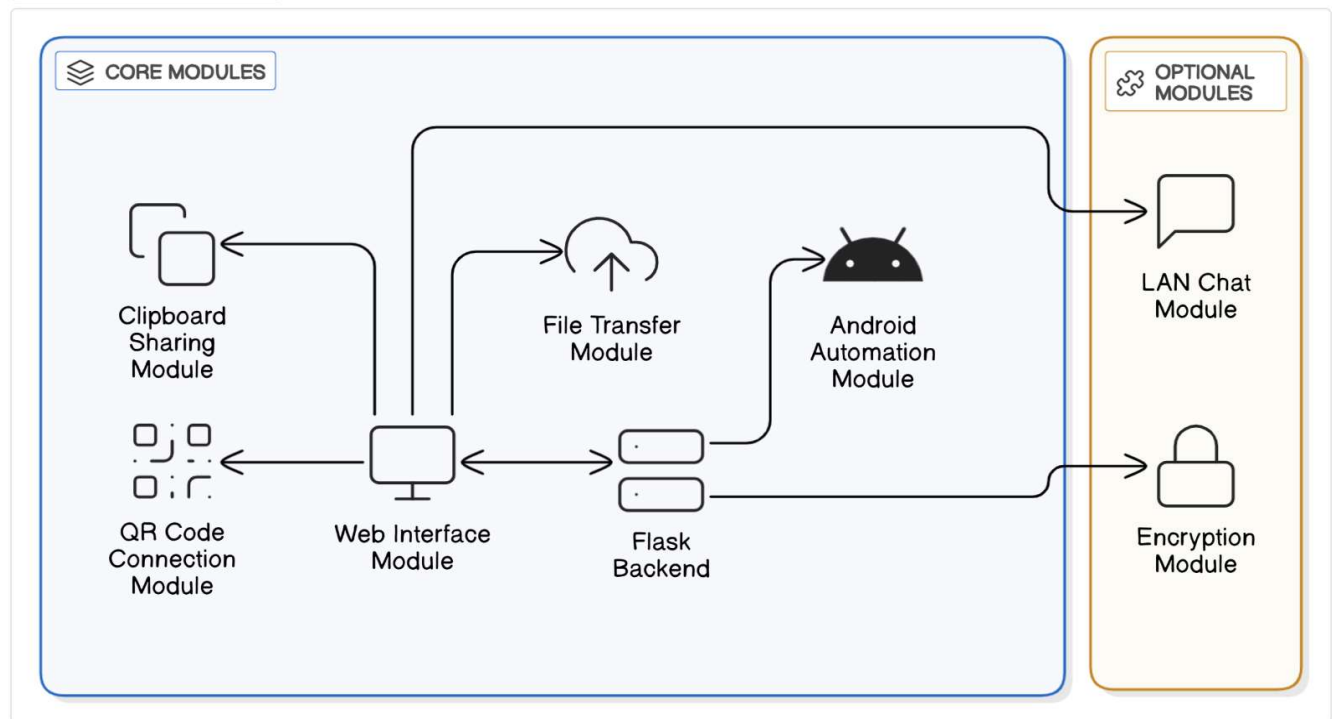
Phase 4: Final Testing & Submission (Weeks 15–16)

All features are tested across devices and browsers. The user interface is finalized with visual polish, animations, tooltips, and guide overlays. Complete documentation, a demo video (optional), and the final project package are prepared for submission.

Testing is done after each task, with focused checkpoints in week 5.5 and week 15. The GUI is progressively developed starting from week 1, with final refinements in the last week.



SYSTEM MODULE:



Core Modules

1. File Transfer Module

- Enables uploading and downloading files over LAN
- Validates file size and type before upload
- Maintains a simple log of each transfer along with metadata

2. Clipboard Sharing Module

- Allows sharing of clipboard text across devices on the same network
- Uses Flask JSON routes and JavaScript clipboard API
- Includes buttons for sending, receiving, and syncing clipboard data

3. QR Code Connection Module

- Generates QR codes that contain server IP, port, and mode (HTTP/HTTPS)
- Helps other devices connect quickly without typing URLs
- QR code is shown on the main dashboard page

4. Android Automation Module

- Designed to run Flask server on Android via Termux
- Uses tools like MacroDroid or Termux Widget for one-tap launch
- Automates opening the browser to the correct local address

5. Web Interface Module (Frontend UI)

- Created using HTML, CSS, JavaScript, and Jinja templating
- Responsive layout for both PC and mobile browsers
- Includes UI for file transfers, clipboard sharing, QR display, and optional chat

Optional Modules

These modules are not critical to the core functionality but add significant value when included:

6. LAN Chat Module

- Enables nickname-based chat within the local network
- Users can join a shared chatroom through the browser
- Built using AJAX polling or WebSocket (optional)

7. Encryption Module

- Adds AES-256 encryption to file uploads for privacy
- Password entered by sender and shared manually or via QR/clipboard
- Decryption is required to download and access the file

REQUIREMENTS:

- Programming Language – Python (Flask Framework)
- Frontend – HTML, CSS, JavaScript, Jinja Templating
- Runtime Environment – Termux (for Android), Python 3.x (for PC)
- Automation Tools (Android) – MacroDroid / Termux Widget / Shortcut Creator
- Browser – Chrome, Firefox, or any modern web browser
- QR Code Library – qrcode, pyzbar, or similar for QR generation and reading
- WebSocket Support – Flask-SocketIO (for optional LAN Chat)
- Encryption Support – pycryptodome (for optional AES-256 file encryption)
- LAN – Wi-Fi or Mobile Hotspot (No Internet Required)

System Requirements:

- CPU – 1.5 GHz Dual Core – ARM (for Android) / Intel/AMD (for PC)
- RAM – 1GB (Minimum), 2GB (Recommended)
- Storage – 100MB for files, logs, and modules
- OS – Android (via Termux), Windows, Linux (PC)
- Network – Stable Local Network (Wi-Fi or Mobile Hotspot)
- Browser Compatibility – Chrome, Firefox, Edge, Brave (mobile and desktop)

LIMITATIONS:

1. **LAN-Only & Offline Use:** Works strictly within the same Wi-Fi or hotspot network with no internet or cloud support.
2. **Manual & Limited Operation:** Requires Termux on Android, manual server start, and browser must remain open for real-time use.
3. **Basic Functionality:** No media streaming or background notifications; optional features like AES-256 encryption and chat may or may not be integrated, depending on available time, development effort, and overall viability

FUTURE SCOPE:

lanvan has great potential for further development and platform-specific improvements.

A major step forward could be building a minimal **Android app** that wraps the Flask backend and web UI into a single package using WebView. This would eliminate the need for Termux and offer smoother startup, background running, notifications, and better file access on Android devices.

To improve performance, **speed optimization** techniques like parallel file transfer, chunked uploads, and browser-side caching can be introduced. At the same time, implementing **user access controls**—such as PIN-protected sessions, role-based file permissions, and trusted device pairing—will enhance security, especially in shared LAN environments.

Looking ahead, **cross-platform expansion** to iOS and desktop via Progressive Web Apps (PWAs) or local installations could make lanvan a universal peer-to-peer tool. With features like encrypted chat history, file previews, and auto-reconnect for chats, lanvan could evolve into a fast, secure, and offline-first file and messaging platform.

CONCLUSION:

LanVan is a solution for those seeking fast, secure, and offline communication in a connected world. It acts as a gateway for seamless file transfer and real-time chat within local networks, eliminating the need for internet access. Our prototype focuses on **browser-based usability**, **smart QR connectivity**, and **speed optimization**, ensuring a smooth and intuitive user experience across devices.

Built using **Flask** for the backend and **WebSockets** for real-time communication, LanVan is engineered to deliver instant messaging and lightning-fast file sharing over LAN. Its architecture supports cross-platform compatibility, allowing users on laptops and Android devices to connect effortlessly within the same network, be it a home Wi-Fi or mobile hotspot.

LanVan introduces a **secure, encrypted communication channel** with optional AES-256 encryption for privacy-conscious users. The system includes a minimal interface with maximum efficiency—requiring no external dependencies or installations on the client side apart from a modern browser. It also integrates a **LAN-scoped QR-based pairing system**, enabling users to quickly connect and verify peers with ease.

From classrooms and offices to remote fieldwork and ad hoc meetings, LanVan empowers users with a robust tool for **collaborative offline networking**. As we continue developing the platform, planned enhancements include **automatic service discovery**, **offline media previews**, and **custom chat channels**, making LanVan a future-ready alternative for LAN-based digital communication.

REFERENCES:

Technical References

1. Flask Documentation – Micro web framework for Python
<https://flask.palletsprojects.com>
2. Termux Wiki – Linux environment and package manager for Android
<https://wiki.termux.com>
3. Python socket Module – Low-level networking interface
<https://docs.python.org/3/library/socket.html>
4. RFC 6455: WebSocket Protocol – Official WebSocket standard
<https://datatracker.ietf.org/doc/html/rfc6455>

Lan Based File Sharing App - Lanvan

5. Flask-SocketIO – WebSocket support for Flask applications
<https://flask-socketio.readthedocs.io>
6. PyCryptoDome – Cryptographic functions including AES-256
<https://pycryptodome.readthedocs.io>
7. QRCode Python Module – QR code generation for connection sharing
<https://pypi.org/project/qrcode/>
8. HTML5 File API – Client-side file handling with JavaScript
https://developer.mozilla.org/en-US/docs/Web/API/File_API
9. JavaScript WebSocket API – Real-time client-server communication in browsers
<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
10. Bootstrap CSS Framework – Responsive front-end design system
<https://getbootstrap.com>
11. Jinja2 Template Engine – Flask’s HTML rendering engine
<https://jinja.palletsprojects.com>
12. MacroDroid Automation App – Automating tasks on Android
<https://macrodroid.com>
13. Termux Widget – Launch shell scripts via Android home screen
<https://wiki.termux.com/wiki/Termux:Widget>
14. Cloudflare – Why HTTPS is Important
<https://www.cloudflare.com/learning/ssl/why-is-http-not-secure/>
15. GeeksforGeeks – Localhost and Networking Basics
<https://www.geeksforgeeks.org/what-is-localhost/>

Source of Inspiration

16. LocalSend – Secure LAN File Sharing App
Google Play Store: https://play.google.com/store/apps/details?id=org.localsend.localsend_app
17. File Manager Pro – Local File Explorer App
Google Play Store: <https://play.google.com/store/apps/details?id=com.folderv.filepro>

